

EE2026 Cheat Sheet
GitHub/ehz0ah
NUS CEG Lee Hao Zhe

Number Systems			
Decimal	Binary	Octal	Hexadecimal
00	0000	00	0
01	0001	01	1
02	0010	02	2
03	0011	03	3
04	0100	04	4
05	0101	05	5
06	0110	06	6
07	0111	07	7
08	1000	10	8
09	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

Decimal → Binary/Octal/Hexa: Division by 2/8/16 (Stop when Q = 0).
↳ Remainders are read from Bottom to Top!!

Binary/Hexa/Octal → Decimal: Compute weighted sum of all digits.
Binary → Hexa/Octal: Group into 4/3 bits (Right to left).

Hexa ↔ Octal: Convert to binary and group into 3/4 bits accordingly.
!! LSB = 1 means the value is ODD. For n bits binary, MAX = 2ⁿ - 1!!

Multiply by 2	Product	Integer Part
0.58 x 2	1.16	1
0.16 x 2	0.32	0

↳ (0.58)₁₀ = (0.10)₂, stop at required dp or fractional part = 0
Decimal → Octal/Hexa: Multiply by 8/16 instead of 2 (binary).

Binary → Hexa/Octal (Fractional): Group fractional 4/3 bits.
↳ Compute weighted sum to get back decimal fractional value.

Binary Arithmetic

A computer may only use adders and shifters to perform all binary arithmetic operations. During subtraction, borrowing 1 from column x and moving it to column x + 1 turns 1 into 2. (Left to Right)

Binary Representation

Signed Magnitude (S-M): MSB is a sign bit, rest of bits = magnitude.

Radix Complement (RC) of n-digit int A with radix (r): A* = rⁿ - A

Diminished Radix Complement (DRC): A* = rⁿ - A - 1.

Binary → 1's Complement: Inverse all bits. (aka DRC)

↳ E.g. 3 + (-2) = 011 + 101 = 001. Bring down carry for addition!!

Binary → 2's Complement: Inverse all bits + 1. (aka RC)

↳ E.g. 3 + (-2) = 011 + 110 = (1)001. Carry is ignored!!

• Magnitude range for all 3: - (2ⁿ⁻¹ - 1) ~ (2ⁿ⁻¹ - 1)
• Positive number is the same for all 3. If MSB = 1, perform below

1's Complement → Decimal: Invert All Bits to get Magnitude, Convert Magnitude to Decimal, Include Negative Sign in Decimal Value.

2's Complement → Decimal: Subtract 1 to get 1's Complement, Invert All Bits and Convert to Decimal to get Magnitude, Add Negative Sign.

SM → Decimal: Ignored MSB, Convert Bits to Decimal, Add - Sign.

Boolean Algebra

• x + 0/ x = x | x + 1/ $\bar{x}$ = 1 | x · $\bar{x}$ / x = 0 | x · 1/ x = x

• De Morgan's Law: $\overline{A + B} = \bar{A} \cdot \bar{B}$, $\overline{A \cdot B} = \bar{A} + \bar{B}$

• Absorption Law: A + A · B = A , A · (A + B) = A

• (2) A + $\bar{A}$ · B = A + B , A · ($\bar{A}$ + B) = A · B

• Logical Adjacency (1): AB + A $\bar{B}$ = A , (A + B)(A + $\bar{B}$) = A

• Consensus Law (3): AB + $\bar{A}C$ + BC = AB + $\bar{A}C$

• Consensus Law: (A + B)($\bar{A}$ + C)(B + C) = (A + B)($\bar{A}$ + C)

Minterm: Product Term, 0 in Truth Table = Complement

Maxterm: Sum Term, 1 in Truth Table = Complement

Canonical Form: All terms in the expression contains all variables in the function. For terms that are missing an element, can multiply with (Missing Element) + (Missing Element Complement), which equals 1.

CSOP → sum the minterms that make output = 1. **CPOS** → product the maxterms that make output = 0. **CPOS = CSOP!!!**

SOP ↔ POS: Apply De Morgan, Expand, Re-Apply De Morgan.

Logic Gates

NAND: Output is False ONLY if both A and B are True.

NOR: Output is True ONLY if both A and B are False.

XOR: Output is True if A ≠ B. **XNOR:** Output is True if A = B.

Critical Path: The longest path (in term of propagation delay) from any input to any output. It limits the speed at which the entire circuit operates.

Positive/Negative Logic

Positive Logic (Active High): High/1 is interpreted as True.

Negative Logic (Active Low): Low/0 is interpreted as True.

Active High ↔ Active Low: x.H = $\bar{x}$.L ↔ x.L = $\bar{x}$.H

Given an equation of SOP, to **design a circuit** made using **ONLY:**

NOR → Create all possible inputs in their original logic. Convert all active low signals to active high. Add gates without simplification.

Change AND gate to OR gate equivalent (Bubbles at input and output of OR). For bubbles at the input, swap connections at the start. Change the last OR gate before output to NOR and add a bubble at the output. Change back the active high signals that were originally active low.

NAND → Same as NOR, just that change OR gate to AND gate equivalent (Bubbles at input and output of AND). Move the bubbles at the input to the output of all the AND gates. EVERYTHING else same.

• Connecting the two inputs of a NOR/NAND gate makes it a NOT gate.
• When speed is critical, prioritise shorter critical path.

• When space/power is critical, prioritise fewer gates.

• Generally, MSOP/MPOS can be created using only NAND/NOR gates.

Modelling Styles in Verilog

Dataflow Modelling: Describes the circuit using continuous assignments ('assign') and logical expressions. Heavy use of Boolean equations and operators rather than defining explicit gates. No explicit use of always block, works with wire data types. Combinational logic.

• Whenever there's an event on RHS, expression is re-evaluated.
• Multiple statements can be executed concurrently.

• Watch out for Multi-Driven Net Error! Order of statements not imp!

Behavioural Modelling: Describes the circuit in terms of sequential and procedural statements. Can describe both combinational and sequential logic. Uses always blocks. 'if-else', 'case', 'for', and 'while' can only be used in always block.

• Anything assigned in an always block must be declared as type reg.
• always block run once when any signal in sensitivity list changes value.
• Statements within always block are executed sequentially. Order matters! begin and end behave like brackets for conditional statements.
• Multiple always blocks run in parallel, watch out for multi-driven nets and race conditions. NO 'assign' in always blocks!!!

Structural Modelling: Describes a circuit by instantiating primitives logic gates or submodules. Interconnects modules using wires.

• Primitive gates' argument takes in output followed by inputs.

• Port Connection by Position: E.g. mux(sel, data, out)

• Port Connection by Name: E.g. mux(.data(data), .select(sel), .out(out))

Simulation/Verification: Process of executing HDL code in a software environment to analyse how a design behaves before synthesizing it into hardware. It helps verify functionality, timing, and logic correctness.

• A testbench is a separate Verilog module that applies stimulus (inputs) to the design and observes the outputs in a simulated environment.

• Simulators usually assume negligible propagation gate delays.

• reg for inputs, wire for outputs, testbench no arguments, run once only.

Logic Minimization

Methods of Simplification: Postulates & Theorems | Karnaugh Map

Karnaugh Map: A graphical method used to simplify Boolean expressions and minimize logic circuits.

• Note that any two adjacent squares differ by **ONLY** one literal!

• Given an equation of SOP, place a 1 in the square that represents a minterm/product term in the given equation. 0 for maxterm/sum term.

• For missing variables in a minterm, it can be 0 or 1. E.g. F = AB + A'BC'. First term AB is missing C. ABC and ABC' are both 1 in k-map.

• n-variables Boolean function has maximum 2ⁿ minterms.

• When grouping, group size should be always 2ⁿ.

• Group should be as **large** as possible (many 1s for MSOP, 0s for MPOS)
• Groups must be either horizontal or vertical (diagonal is invalid) but can wrap around edges of the map.

• The variable that changes value in the group is eliminated, or the variable that doesn't change value in the group remains
• A 1 may be circled multiple times if doing so allows fewer circles to be used. **Minimum essential** number of groupings to cover all the 1s.

• For MPOS: Group the 0s instead!! Every group will give you a sum term instead of product. Look at the unchanged variables, invert them if their values are 1 and keep the variable if 0.

• Don't-care condition is marked with X in K-Map, X = 0 or 1.

Combinational Blocks

Combinational blocks are predefined logic circuits made up of combinational logic (AND, OR, NOT, XOR, etc.) that do not have memory or feedback loops. The output of a combinational block depends only on the current inputs and not on past inputs.

Multiplexer (MUX): A MUX is a combinational circuit element that selects data from one of 2^N inputs and directs it to a single output, according to an N-bit selection signal.

Demultiplexer (DEMUX): A DEMUX connects an input signal to any of 2^N output lines, based on an N-bit selection control.

Decoder: N-bit input, 2^N outputs. Converts binary input into ONE active output. Multiple outputs but only ONE active at a time.

Encoder: 2^N inputs, N-bit output. Converts ONE active input into binary output. Multiple inputs but only ONE is active at a time.

Priority Encoder: Encoders with a hierarchy of importance. If multiple inputs are active (1) at the same time, it prioritizes the highest-order (MSB) input and ignores the others (Mark as Don't care-X).

Adders: Adders are fundamental combinational circuits used to perform binary addition. They are categorized into three types (one-bit):

Half Adder → Adds two bits (no carry-in). Outputs sum + carry-out.

Full Adder → Adds three bits (includes carry-in). Same output as half.

Parallel Adder → Cascades multiple full adders for multi-bit addition.

• In general, n full adders need to be used to form an n-bit adder.

↳ Carry Ripple Effect: Output of each full adder is not available until the carry-in from the previous stage is delivered. Carry bits have to propagate from one stage to the next as the carries ripple through the carry chain → also known as ripple carry adders.

↳ This slow rippling effect is substantially reduced by using carry look ahead adders.

Magnitude Comparator: It has 3 outputs (A > B, A < B, A = B).

↳ It compares two n-bit binary numbers (A and B) and determines A & B relationship and outputs a 1 at the correct output.

Tri-State Logic Element: It is a digital circuit component that has three possible output states, Logic High, Logic Low, High Impedance (Z).

↳ This allows multiple devices to share the same output line without interference.

↳ Only one device should write to the bus at a time, the others must remain in a high-impedance (Z) state.

↳ Commonly used when multiple devices need to share a single data bus.

Combinational Logic vs Sequential Logic

Combinational Logic Circuits: Outputs depend on current inputs.

Sequential Logic Circuits: Outputs depend on **current and previous** inputs → Memory! Requires separation of previous, current, future → states! 2 Types of sequential circuits → Synchronous (Clocked): responds to inputs at discrete time instants governed by a clock input. Asynchronous (Un-clocked): Responds whenever input signals change.

Set-Reset Flip-Flop (SR-FF) (ASYNCHRONOUS)

• FF can store 1 bit of information. Most basic FF: Set-Reset FF/Latch
• Usually implemented with NOR / NAND gates

• For SR FF, S is to set output Q = 1, R is to reset Q = 0, S = R = 0 is to hold the output Q = Q⁺, S = R = 1 is an invalid state as Q = Q⁺.

• Output configured as input (Feedback) creates memory.
• Switch debouncing is a common use of S-R FFs.

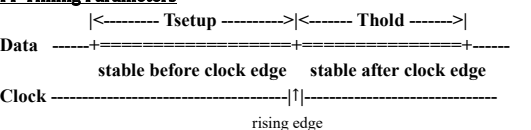
Jack Kilby Flip-Flop (JK-FF)

• The JK flip-flop is an advanced version of the SR flip-flop that **eliminates** the "invalid" state in the SR flip-flop. Combines the functionality of the SR flip-flop (J for Set and K for Reset) with the additional ability to **toggle** in replace of invalid state.

• The JK FF is a **synchronous** circuit → respond only at the clock edges
↳ It can be positive or negative edge. Look closely at the clock input.

• JK FF can be combined with **asynchronous clear (CLR)**. A high in CLR will clear output to 0 regardless of other inputs or the clock.

FF Timing Parameters



TpHL: Time taken for FF output to change state from **High to Low**.

tpLH: Time taken for FF output to change state from **Low to High**.

↳ If input changes state right at the active clock transition, the output will be unpredictable. Thus, input changes must meet required **setup & hold** times of device == Operating Speed of device.

Other Types of FF

D-FF: The output Q follows the input D at the rising clock edge.

T-FF: The output toggles (flips its state) when T = 1 on the rising clock edge. If T = 0, the output remains unchanged.

D-Latch: The output Q follows the input D only when the enable signal (EN) is active (1). When EN = 0, the output holds its previous state.

↳ T-FF are easy to construct from other FFs, hence they are not often used commercially. You can make a T-FF from a JK-FF by connecting one T signal to both J and K.

↳ Connecting N number of D-FF together will cause the Nth D-FF to have a change in output N - 1 cycle of delay later in response to a change in input for the first D-FF. Adding one flip-flop = Add one cycle of delay.

Counters (Asynchronous Ripple Counters)

• Counters are commonly built using FFs. For a counter to count to 15, it requires 4 bits and that means 4 FFs. The **output** of one FF is the **input CLOCK** of the next FF. Each FF output is half of their input clock frequency (F/2 successively). First FF is the LSB and the last FF is MSB.
• Since FFs are stacked and earlier FFs output is input clock for later FFs, **asynchronous** here means circuit elements (FFs) do **not** get the clock input **simultaneously**. Opposite for synchronous.

State Transition Diagram for Counters

• Circuit can only be in one state at any time
• Circuit is in each state for one clock cycle

• Arrow indicates state transition at active clock edge (Falling for ripple)

Counters : Mod - X

• X **distinct** count states → Mod - X Counter

• N x FFs connected output to input clock will have 2^N states (Mod-2^N)

How to reset a N x FFs counter at X before reaching 2^N state?

- Force it to reset early by using FFs with CLR functions to reset the FF regardless of clock or inputs. Assume counter starts from 0.
- Next, determine the binary representation of X that you want. E.g. X = 10 = 1010. From here, we can tell that Q1 (first bit) and Q3 (third bit) outputs are high. Feed Q1 and Q3 into a NAND gate. Connect the output of the NAND gate to **asynchronous CLR**. When the counter reaches 1010, the NAND gate outputs 0, triggering CLR and resetting the counter to 0000.

• This method is applicable for designing counters with any desired modulus X that is less than the maximum possible states 2^N.

NOTE: To count down instead of up, connect output complements (Q') of FF to clock inputs of succeeding FFs instead of the usual output (Q).

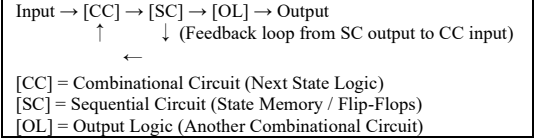
Limiting Frequency of Asynchronous Ripple Counters

- Due to propagation delays of the FFs in the chain adding up, **clock input** of the Nth FF changes only after t_o + (N – 1) * Δt_{pd}.
- **Output** of the Nth FF changes only at N * Δt_{pd} after t_o.
- In order for counters to not break down, the propagation delay added up CANNOT BE longer than the period of the very first clock input.
- T_{clock} ≥ (N * Δt_{pd}) and f_{max} ≤ [1 / (N * Δt_{pd})]
- If the above conditions are violated, the ripple counter will skip / fail to reach certain states, causing timing errors.

Synchronous (Parallel) Counters

- All FFs are driven by the **same clock signal simultaneously**, rather than relying on the output of the previous FF as a clock.
- This **removes** the cumulative propagation delay issue and ensures that all **outputs** change **simultaneously**.
- Output (Q) of the FFs are fed into a combinational circuit where the output of the circuit are the inputs to the sequential circuit.
- Critical Path Delay = Δt_{pd} (FF) + Δt_{pd} (Gates of CC)
- ↳ Since it is parallel, choose only 1 FF and the worst CC path.
- Operating frequency is irrespective of the # of FFs but need more gates.
- For counting down, Q' is feed into the combinational circuit instead.

Digital System



Flip-Flop Design (Without Output Logic)

1. First step is always to come up with the characteristic table.
↳ Includes inputs to CC and outputs Q & Q+ (Can be multi-bits!)
2. If tasked to use a type of default FF (D or JK), combine the characteristic table with the excitation table of the default FF.
3. From the combined table, remove Q⁺ as combinational circuits only depends on its current inputs.
4. Perform K–Map simplification for each output to get CC.

Counter Design (Synchronous/Parallel) (Without Output Logic)

1. Draw a simple state transition diagram for the unique count states.
2. Construct the functional block diagram by understanding how many bits is required for the maximum count value. # of bits = # of FFs required in SC = # of inputs to SC = # of outputs of sequential circuit connected as inputs to CC. Add extra inputs (extra logic) to CC if any.
3. Create the next state table (Current state Q and next state Q⁺)
↳ Add the extra inputs to current state in the next state table if any.
4. If tasked to use a type of default FF (D or JK), combine the next state table with the excitation table of the default FF.
5. From the combined table, remove Q⁺ as combinational circuits only depends on its current inputs.
6. Perform K–Map simplification for each output to get CC.

Output Logic

- Suppose we are designing a Mod–3 Counter and each unique count state requires 32 bits. Without output logic/FSM, 32 FFs are required to make this counter. With output logic, only 2 FFs are required for 3 states.
- Without FSM/output logic, more FFs but lesser gates are required. This will result in an increase in maximum operating speed of the counter.
- Coming up with the **CC** for the output logic is very simple.
↳ First, create an output logic table that **maps states to outputs**. Every output inside the “bubble” should be shown here (Separate into bits).
↳ Perform K–Map simplification for each output to get CC.
- If the MSOP of each output is **purely functions of the current state** and NOT dependent on the system inputs, it is a **Moore Machine**.
- There should not be “other states” in output logic table.
- RESET in OL is asynchronous, RESET in SC & CC is synchronous.
- RESET in OL DO NOT affect state, just overriding the outputs.

Finite State Machine (FSM)

- Synchronous machines with “states” of operation.
- At each active clock edge, combinational logic computes outputs and next state, as a function of inputs and present state.

Mealy Machine: Output is a function of **present state** and **inputs**.

- ↳ Output changes synchronously (due to state) and asynchronously (due to inputs that are not clocked). Inputs are also found at the output logic.
- Moore Machine:** Output is a function of **present state** only.
↳ Output only changes synchronously.

- In FSM, circles/bubbles represent states, where each state specifies values for all outputs.
- Arcs represent transitions between states, and they are labelled with inputs that triggers the transition on the active edge of the clock.
- RESET arc from outer space indicates initial state upon reset.
- Moore Machine typically has **more states** than Mealy Machine.

Modular Design of State Machines

- Main Machine: Executes main algorithm, controls the submachines & get the job done. Commands & gets feedback signals from submachines.
- Submachines respond to external inputs & commands from main machine. Can give outputs as well as feedback to the main machine.
- Common examples of submachines are counters, shift registers, etc.
- Often, the main machine is called the **controller**, and the submachines are called controlled circuit elements or **architectural elements**.
- Trick here is to **modularize** appropriately and pick suitable components for the submachines that simplify the design problem.

Field-Programmable Gate Array (FPGA)

- An FPGA is a **reconfigurable** semiconductor device that contains an array of programmable logic blocks (such as look-up tables, flip-flops, multiplexers) and a hierarchy of interconnects that can be configured by the user to implement **custom** digital logic circuits.
- LUT:** Implements any N-input logic function (typically 6-input → 64-entry truth table). Think of it as a programmable combinational gate.
- Flip-Flop:** Stores a single bit. Used to store state in FSMs, counters, pipelining.
- Block RAM (BRAM):** Prebuilt dual-port memory blocks inside FPGA. Used for register files, video framebuffers, stack, FIFO queues.
- DSP Slices:** Pre-built multiplier-accumulator units. Great for signal processing: filters, FFTs, convolution. Specialize for fast multiplication.
- Routing Fabric:** A massive interconnect network that connects LUTs, FFs, BRAMs. Toolchain decides best wiring paths based on logic and constraints.
- How it works:** Write Verilog to describe behaviour. The synthesis tool turns it into gates and wires (**netlist**). The place & route tool maps that netlist into FPGA fabric. The **bitstream** is downloaded to configure the FPGA hardware. Unlike software, your Verilog code doesn't "run". It becomes a circuit.

RISC–V Microprocessor

- RISC-V is an open-source **Instruction Set Architecture (ISA)** that defines a **minimal, modular, and extensible set of machine instructions** intended for a wide range of computing systems, from embedded microcontrollers to high-performance processors.
- RISC = Reduced Instruction Set Computer (few, simple instructions).
- All instructions are **fixed length 32 bits** (unless compressed).
- Operates on a **load-store architecture**: only lw/sw access memory.
- Simple instruction formats: R, I, S, B, U, J types.
- What is in a basic RISC-V core?**
- PC (Program Counter): Points to current instruction.
- Register File: 32 registers x0–x31 (all 32-bit). x0 is hardwired to 0.
- ALU: Performs arithmetic, shifts, and comparisons.
- Instruction Memory: Stores your program (mapped to BRAM in FPGA).
- Data Memory: For load (lw) and store (sw) instructions.
- Control Unit: Decodes instruction and generates control signals.
- Immediate Generator: Extracts immediate values from instruction.

RISC–V 5 Stage Pipeline

- **IF (Instruction Fetch):** Instruction is fetched from instruction memory at PC. PC typically increments by 4 (next instruction).
- **ID (Instruction Decode):** Instruction is parsed: fields like opcode, rd, rs1, rs2, funct3/7 are extracted. Register file is accessed (rs1, rs2 read). Immediate value is generated (I, S, B, U, or J format).
- **EX (Execute):** ALU performs operation: add, sub, slt, and, or, branch target, etc. For branches: comparator checks condition (beq, bne), target PC computed. For memory access: ALU computes effective address.
- **MEM (Memory Access):** Access data memory if needed: lw loads data from memory to pass into WB. sw stores rs2 into memory.
- **WB (Write Back):** Result is written to register rd, either from ALU (arithmetic instructions) or from memory (load instructions).

Further Information

- Not all instructions use all 5 stages
- RISC–V = simple, flexible, FPGA-friendly
- ARM = efficient & mature in embedded/mobile
- x86 = complex, powerful, but less ideal for new hardware

Verilog

Hardware Description Languages (HDLs) are programming languages for describing digital circuits and systems. Verilog code is used to describe RTL (Register Transfer Level) designs. Used for Modelling, Simulation and Synthesis of digital circuits.

- **Net/wire:** 3 basic values – 0, 1, Z (High Impedance). Do not store value as it's determined by its driver. No driver = Z. Use `assign`. Wire CANNOT be assigned values within an always block!
- **reg:** Can store value. Uninitialized = X (Unknown). Can be used to model both combinatorial and sequential logic. Cannot use `assign`, need use procedural assignment (always @). Default is 32-bits.
- **parameter:** Parameters are constants in Verilog (hence illegal to modify their value at runtime). They are often used to customize modules, helping to create more reusable code.
- **Verilog Bit-wise Operator Precedence:** ~ > & > ^ > |
- **always** block runs once when sensitivity list changes value. All signals have to be posedge or negedge if one signal is posedge / negedge in the sensitivity list.
- While code inside an always block appears to execute sequentially, the type of assignment determines whether updates occur immediately (sequentially) or at the end of the time step (concurrently).
- **Blocking Assignments** (=) execute sequentially, meaning each statement updates immediately, and subsequent statements use the updated values.

- **Non-Blocking Assignments** (<=) execute concurrently, meaning updates are scheduled to occur at the end of the time step, so subsequent statements use the old values.

FDXX Primitives in 7series FPGA:

- FDCE: D flip-flop with clock enable and asynchronous clear.
- FDPE: D flip-flop with clock enable and asynchronous preset.
- FDRE: D flip-flop with clock enable and synchronous reset.
- FDSE: D flip-flop with clock enable and synchronous set.

↳ FDRE # (.INIT(1'b0) FDRE_inst (.Q(Q),.C(C),.CE(CE),.R(R),.D(D))
FSM: Generally, always @(*) is used for CC, always @(posedge clk) for SC, assign statements for OL.

- Common Gates Boolean Representation

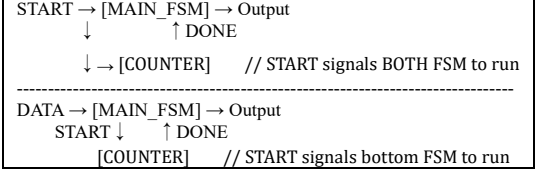
A ^ B = A'B + AB' // XOR or A ⊕ B
~(A & B) = A' + B' // NAND
~(A + B) = A'B' // NOR
(A ^ B) = AB + A'B' // XNOR or A ⊙ B

Additional Notes

- 1s complement need shift and add carry while 2s complement no need.
- NAND and NOR are universal gates.
- Bubble Pushing is done using De Morgan's Law.
- Concatenation: {A , B} | Replication: {nA}
- always block CAN be used in testbench code.
- Multiplexers can be used as lookup tables (LUT) to perform logic functions! Just put the output values in truth table as inputs to MUX.
↳ Inputs in truth table will act as the select bits for the MUX.
- To code Priority Encoder, use nested conditional operators, starting from MSB and progressively move to the LSB.
- Most of combinational blocks can be code with just ternary operators.
- Output logic is just instead of using the actual output values like count as states, you create new states variables like S0, S1, etc for lesser bits.
- If a counter requires N bits but does not count all possible 2^N states, the “other states” could be set to X (**Minimum Cost**) or default state (Whatever the default state is, **Minimum Risk**).
↳ If “other states” is used for K–Map simplification so that only 1 bit is changing at a time, mark them as X or default state on the K–Map.
- If needed an input at t – 1, can use any FF to delay by 1 clock cycle.
- **Simple** FF for SC results in a **more complex** CC.
- For D–FF, excitation table = characteristic table.
- Excitation Table for JK–FF

Q	Q ⁺	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

- Characteristic Table for JK–FF can be determined from excitation table.
- Separate your counter block from your FSM! The main FSM should show the transition of the main logic of the system.



- When an output has to be asserted as soon as the conditions are satisfied, the outputs should have asynchronous updates. Outputs are functions of inputs (Mealy Machine).
- To convert Moore Machine to Mealy Machine, instead of tying output to the state, tie it to the transition. It is reflected using a / symbol.
↳ Output can also be inside the “bubbles” but they are represented as functions of inputs.
- Outer space arrows are synchronous and is checked in every state.